Introduction to Computer Networks and the Internet
COSC264

Network Programming with Sockets

Slides prepared by [1]Dr.-Ing. Andreas Willig
Updated by [2]Dr. Barry Wu

[1]Department of Computer Science & Software Engineering
[2]Wireless Research Centre
Faculty of Engineering

Semester 2, 2026

UC UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

Scope

Goals:

- Explain the concept of sockets
- Introduce the socket API
- Show socket programming examples

Important

You can use the socket interface without understanding the operation of the underlying protocols (TCP, UDP, IP). What matters are the services offered by these protocols!

Outline

1 Preliminaries

2 Sockets Fundamentals

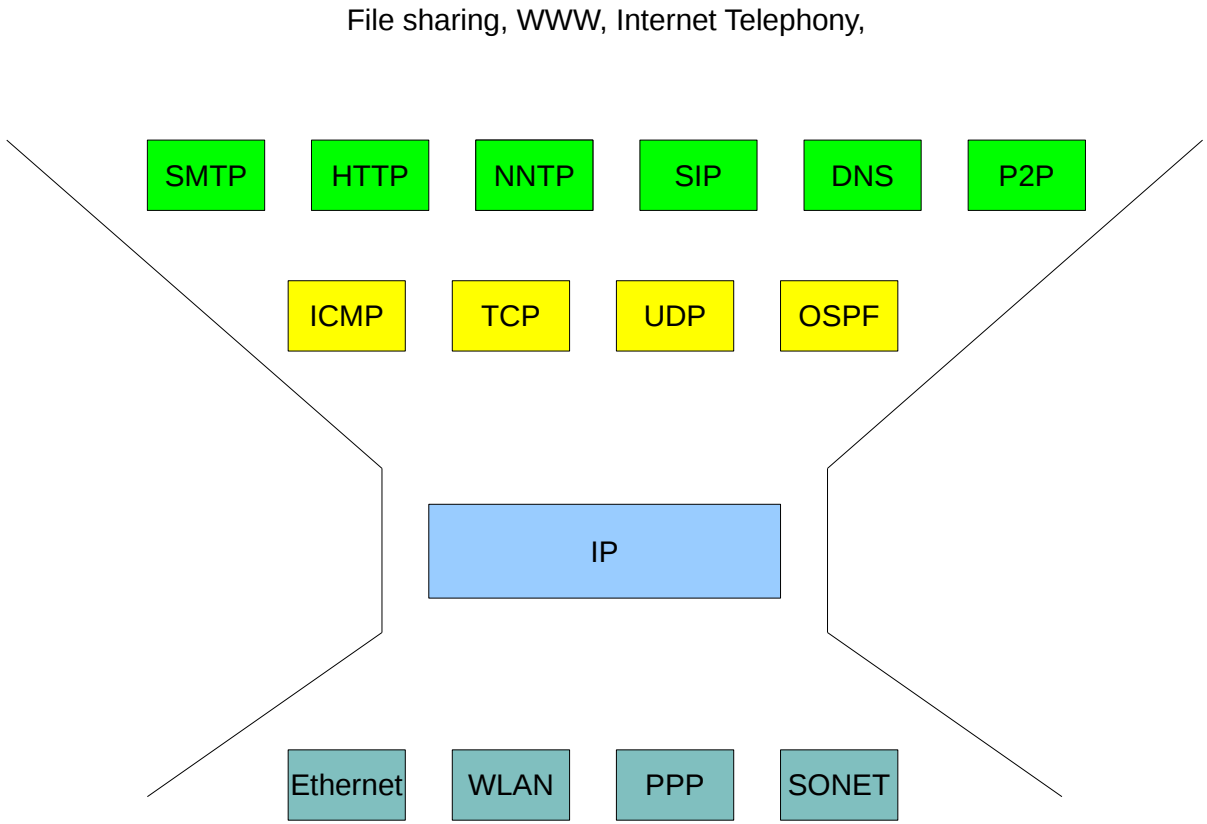
3 The Socket API

4 Socket Programming Examples

Outline

1. Preliminaries
   - IP Protocol
   - UDP and TCP
   - Processes and Blocking
2. Sockets Fundamentals
3. The Socket API
4. Socket Programming Examples

The Role of IP in the Internet Protocol Stack



- IP = Internet Protocol
- There are two protocol versions (4, 6), here we use IPv4
- “Everything over IP, IP over everything”

IP Addresses

- Each host is identified by one or more **IP addresses**
  - A host has as many IP addresses as it has network adapters
  - End hosts usually only have one IP address
- The IP address not only identifies the host, but also helps the network to find a path to this host
- Humans normally do not work with IP addresses directly but with human-readable host names like `www.canterbury.ac.nz`
- There exists a special service / protocol called the *domain name service* (DNS) which translates human-readable host names to IPv4 addresses, the Internet itself only works with IPv4 addresses
- We ignore IPv6 in the following discussion

IP Address Representation

- IPv4 addresses have a width of 32 bits
- They are supposed to be worldwide unique
- IP addresses are written in **dotted-decimal notation**, e.g.:

  `130.149.49.77`

  where decimal numbers are separated by dots

IP Service – Best Effort

- Basic IP service is **packet delivery**
- This service is:
  - Connectionless: no connection or shared state is set up before datagram delivery starts
  - Unacknowledged: IP does not use acknowledgements
  - Unreliable: on IP level no retransmissions are carried out
  - Unordered: IP does not guarantee in-sequence delivery [1]
- It is very similar to sending postcards
- This kind of guarantee-nothing service is called **best effort**

Outline

1. Preliminaries
   - IP Protocol
   - UDP and TCP
   - Processes and Blocking
2. Sockets Fundamentals
3. The Socket API
4. Socket Programming Examples

UDP and TCP

- In the Internet there are two further foundational protocols:
  - UDP – User Datagram Protocol
  - TCP – Transmission Control Protocol
- Both protocols operate *on top* of IPv4, they generate packets and stuff these as payload data into IPv4 packets
  - This is related to **protocol layering**
- UDP or TCP sessions run between a pair of IP addresses
- They add own addressing capabilities (*port numbers*); Analogy:
  - An IPv4 address corresponds to the street address of a house
  - A port number refers to an individual person in that house
- In a networking context:
  - An IPv4 address refers to an end host
  - A port number refers to a particular application / server software running in that end host (out of possibly several)

UDP

- UDP = User Datagram Protocol
- UDP offers a service that is similar to the service of IPv4:
  - Connectionless
  - Unacknowledged
  - Unreliable
  - Unordered
- Almost the only thing that UDP really adds are port numbers

TCP

- TCP = Transmission Control Protocol
- The TCP service is:
  - Connection-oriented
  - Reliable and in-sequence
  - Byte-stream oriented
  - Full-duplex
- In other words: using the TCP protocol allows for “safe and reliable” data transfer over the Internet, where a string of bytes is transferred reliably and in the right order
- To do this, TCP does a lot of work under the hood . . .

More on Port Numbers

- Port numbers are local to a host, used to address different applications / processes / threads
- Some basic rules:
  - A port number is a 16 bit long unsigned integer
  - A process on a host can allocate one or more port numbers
  - One port number is allocated to at most one application / process
- Some port numbers are often allocated to well-known applications / higher-layer protocols (**well-known ports**), e.g.:
  - Port 80: HTTP (World Wide Web, runs over TCP)
  - Port 22: SSH (Secure Shell, runs over TCP)
  - Port 25: SMTP (Email, runs over TCP)
  - Port 53: DNS (runs over UDP or TCP)
- **But**: this is pure convention, not enforced!

Outline

1. Preliminaries
   - IP Protocol
   - UDP and TCP
   - Processes and Blocking
2. Sockets Fundamentals
3. The Socket API
4. Socket Programming Examples

Processes and Blocking Socket Calls

- Some socket calls (e.g. `accept()` or `select()`) have an important property: they can be **blocking**
- This refers to the concept of processes in operating systems (OS) like MS Windows / macOS / Linux
- A process is a running instance of a program / application
- Operating systems create an abstraction for processes that:
  - makes them believe they have their own exclusive processor
  - allocates memory exclusively (not accessible by other processes)
  - can request other OS resources like sockets, pipes, files, etc

Processes and Blocking Socket Calls (2)

- In reality the OS uses a time-sharing approach to let several processes run on one processor
- Processes can be in one of several states, these include:
  - Running: actually running on the processor
  - Runnable: ready to run on a processor, but currently not doing so
  - Blocked: process is waiting for input from a hard disk / socket / ...
    – when input arrives the process becomes runnable or running
    – while blocked, the process does **not** use any CPU cycles
- The system call `select()`:
  - Waits for input on one or more sockets (e.g. incoming packets)
  - If no input is available at time of call, the calling process will be put into blocked state, hence does not use the CPU
  - This is better than the alternative called **busy-looping**, in which a process repeatedly checks the sockets in a loop to see whether new data has arrived – this consumes a lot of CPU resources

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API

4 Socket Programming Examples

Introduction

- The socket API was introduced with 4.2BSD Unix in 1983
- It follows the Unix philosophy that (almost) everything is a file or could be made to look like a file, programmers can use sockets in a way similar to using files
- All Internet applications like web, mail, FTP etc. work with sockets

Ports and Sockets

- Applications use the programming abstraction of a **socket**
  - A socket is bound to exactly one port
  - A socket is associated with underlying protocol (often UDP or TCP)
    - Note: the application does not need to know how or when the protocol operates precisely, but it needs to know the *service*!
  - A socket is associated with **buffers**:
    - These buffers decouple the application from the underlying protocol
    - Receiving UDP / TCP protocol entity places incoming data into receive buffer, applications `recv()` data from buffer at their leisure
    - Application at transmitter `send()`'s data into transmit buffer, TCP/UDP entity sends it at its discretion
  - It is possible to both transmit and receive from a socket
- Conceptually, a socket has similarities with file handles

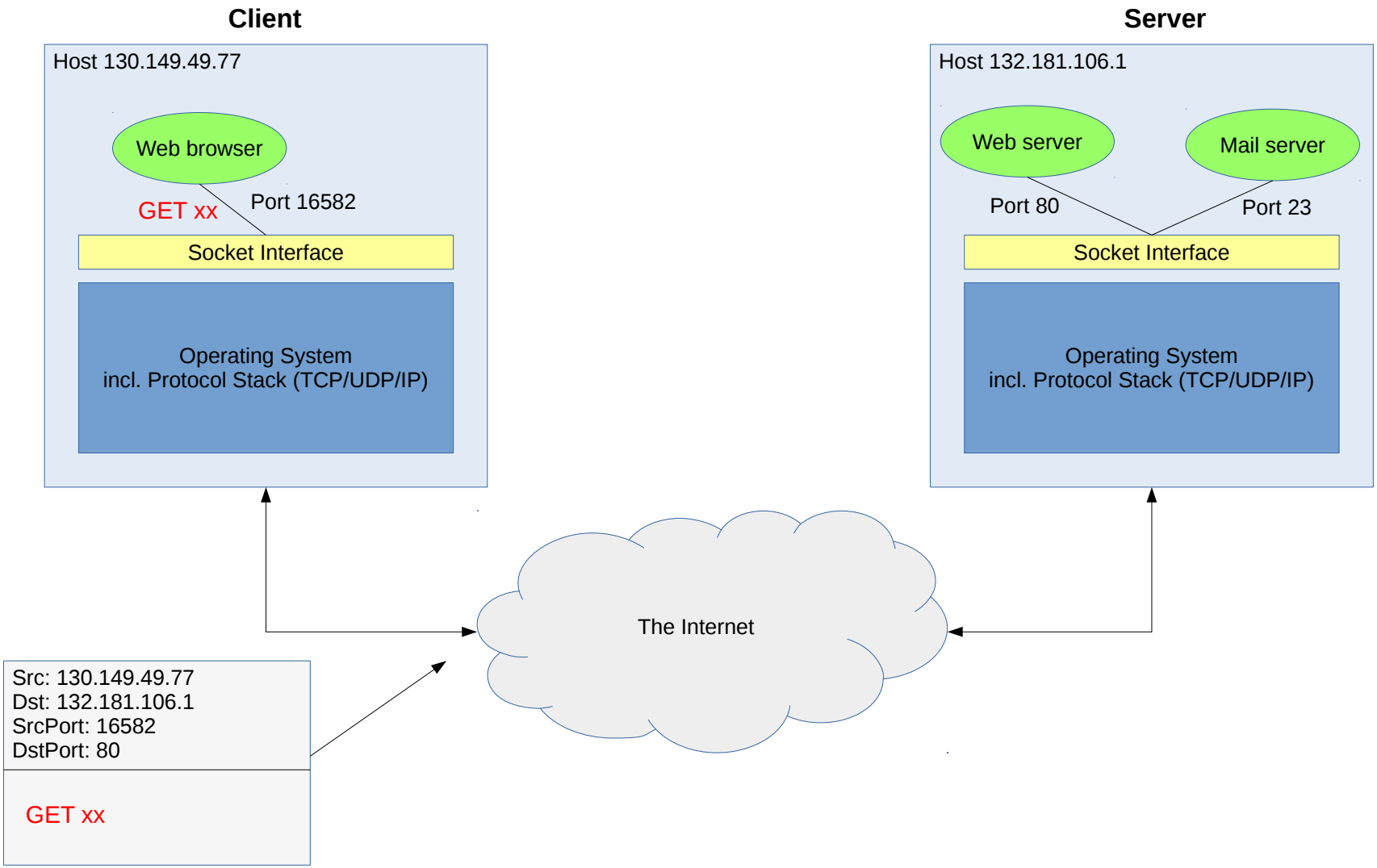
Ports and Sockets (2)

- For sending over a socket you can call `send()` or `write()`
- For reading from a socket you can call `recv()` or `read()`
- When writing to a socket:
  - UDP: the data is encapsulated in UDP datagram and transferred – each `send()` leads to a separate datagram
  - TCP: the data is buffered and possibly combined with data from further `send()`'s before transmitting
- **Careful**: a successful `send()` call does not mean that the data has already been successfully received, only that the data could be placed in the sender's socket buffer!
- The socket API has some overlap to the Unix file API; Sockets can be used the same way as file handles e.g. in `select()`

Client/Server Paradigm in the UDP/TCP/IP Context

- Client applications:
  - run on an end host
  - A client needs to know servers IP address / port number, needs to have an own IP address / port number and a socket bound to these
  - It then initiates contact with the server “through its socket”
  - Afterwards a client application can end
- Server applications:
  - run on an end host
  - need an IP address / port number that is known to clients and need to have a socket bound to these
  - accept service requests from clients and respond to them
  - usually run all the time and support several clients
- Client and server applications can even run on the same machine and communicate through the socket interface
  - Client uses special address `127.0.0.1`, known as **localhost**

Client/Server Paradigm in the UDP/TCP/IP Context (2)



Socket Types

- When a socket is created, it can be of one of several types
- A **stream socket**:
  - Offers reliable, in-sequence delivery of a byte-stream
  - Is often used with the TCP protocol
  - If delivery fails for some reason, the sending application is notified
- A **datagram socket**:
  - Offers delivery of distinct finite-sized datagrams
  - Does not guarantee reliable or in-sequence delivery
  - Is often used with the UDP protocol
  - If delivery fails the sending application is not informed
- These are the main types, there are other types:
  - **Raw sockets**: bypass UDP or TCP and directly send IP packets
  - **Sequenced-packet sockets**: a variant of a stream socket
  - Further types might be available depending on OS

Socket Types (2)

- Datagram sockets offer a datagram service
  - when `send()`ing *x* bytes to a datagram socket the written data is encapsulated whole into a UDP packet and sent out
  - the receiver will `recv()` a block of at most *x* bytes from its socket
- Stream sockets offer a **byte-stream service**:
  - Smallest unit of transmission is one byte
  - Different `send()`'s do not necessarily lead to different packets
  - Receiver cannot detect which was last byte supplied at the transmitter with one `send()` command, e.g.:
    - Sender performs `send()` with ten bytes, followed by `send()` with ten bytes, followed by `send()` with 20 bytes
    - Receiver `recv()`s one piece of 40 bytes and cannot tell how many `send()`s contributed to it, nor where boundaries were
- In other words: datagram sockets preserve record boundaries, stream sockets do not
- Sequenced-packet sockets preserve record boundaries

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API

4 Socket Programming Examples

Outline

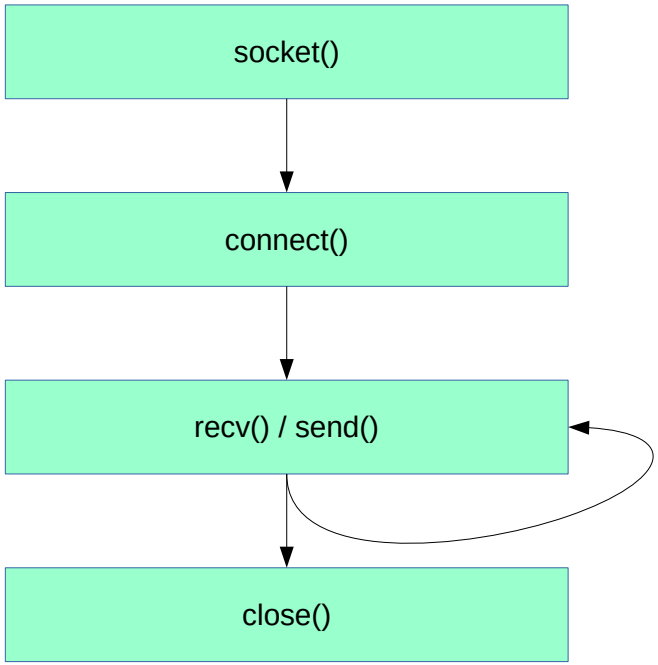
1 Preliminaries

2 Sockets Fundamentals

3 The Socket API
- Overview
- Main Socket API Functions
- Helpers

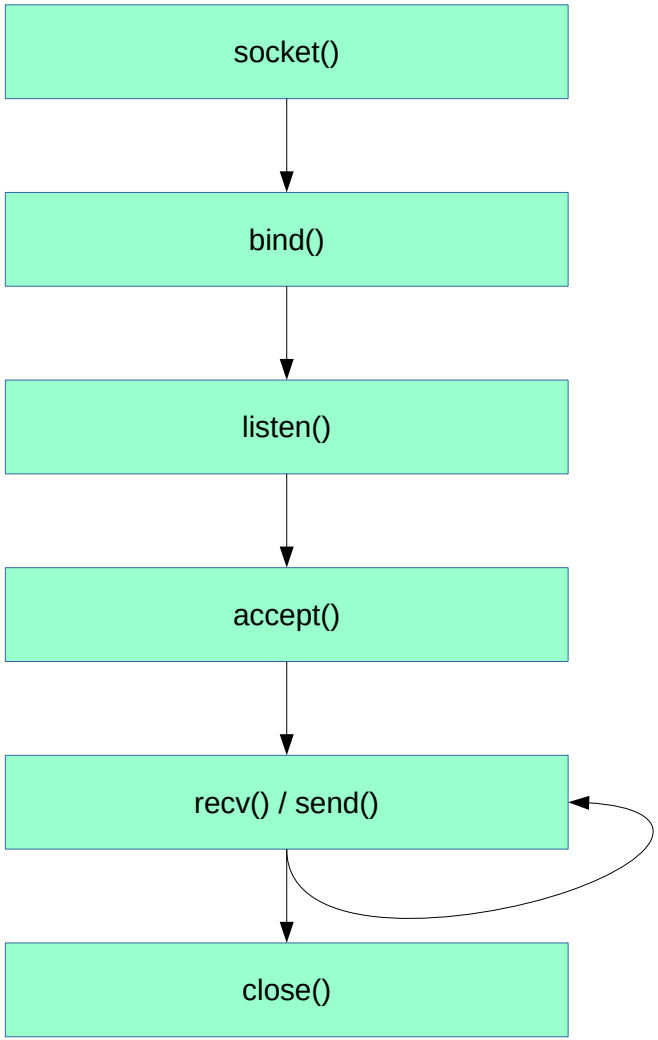
4 Socket Programming Examples

Example Workflow: TCP Client



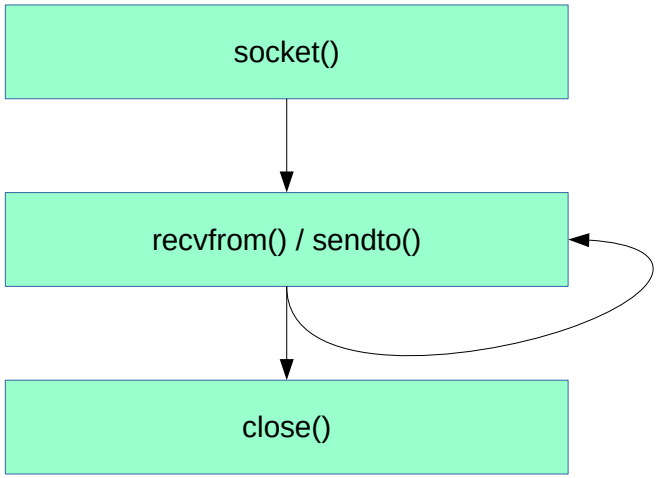
- `socket()`: create a socket, this allocates resources (e.g. buffer, socket handle), and assigns a **random unused port number**
- `connect()`: establish connection with server
- `recv()`/`read()`/`send()`/`write()`: reading from / writing to socket
- Alternatively one can also use `sendto()` and `recvfrom()`, to specify additional parameters
- `close()`: close connection

Example Workflow: TCP Server



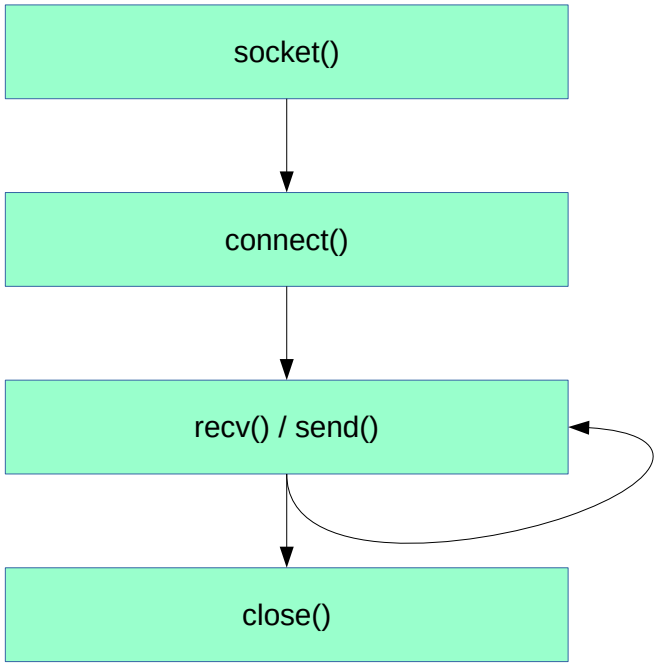
- `bind()`: bind a socket to a particular port number and IP address
- `listen()`: declare willingness to accept incoming connections, allocate resources (queue) for incoming connection requests (TCP)
- `accept()`: accept an incoming connection request (i.e. take it from the connection request queue) and **create a new socket** for this connection, bound to the same port as the socket we called `listen()` on

Example Workflow: UDP Client



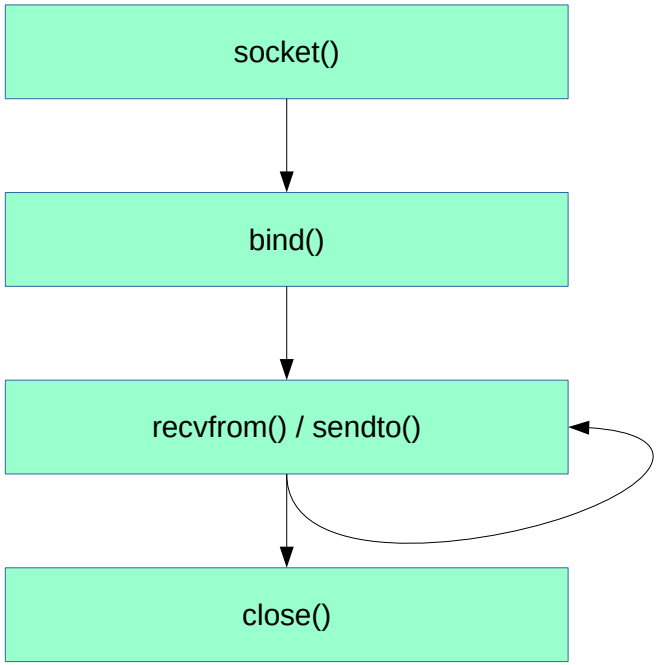
- Compared to TCP client no `connect()` system call is necessary
- Receiver address then has to be supplied to `sendto()`
- `recvfrom()` receives data and returns address of transmitter

Example Workflow: UDP Client with `connect()`



- The `connect()` system call supplies a default receiver address
- `connect()` can be called multiple times to change the default receiver
- `send()` then sends to the last receiver specified with `connect()`

Example Workflow: UDP Server



- Compared to TCP server no `listen()`, and `accept()` calls are necessary

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API
- Overview
- Main Socket API Functions
- Helpers

4 Socket Programming Examples

Main Socket API Functions

- We only discuss the most important socket calls
- We use the Python 3 socket API as a guideline, but we will not discuss all available options
- The Python API closely wraps the Windows and Unix socket APIs, most low level networking APIs do this, they contain the same basic functions and follow the same workflow
- You can find the documentation in the Python socket page: `https://docs.python.org/3/library/socket.html`
- All socket calls throw exceptions derived from Python's `OSError`, these include a string indicating the cause of the error, the exceptions that can be raised by a particular socket call are described in the socket module docs

socket()

```
from socket import socket

socket(family=AF_INET, type=SOCK_STREAM, proto=0, fileno=None)
```

- Class constructor; creates a new socket object, including allocation of resources like the socket buffer, assigns a random un-used port number to it, and returns a socket object
- Parameter `family`:
  - Selects the protocol / address family to be used for the socket
  - Use `AF_INET` for IPv4 (default value), `AF_INET6` for IPv6
  - Others available, depending on OS (Linux: see `man socket`)
- Parameter `type`:
  - Indicates socket type
  - Options include:
    - `SOCK_STREAM` for stream sockets (use this default option for TCP)
    - `SOCK_DGRAM` for datagram sockets (use this option for UDP)
    - `SOCK_RAW` for raw sockets

socket() (2)

```
from socket import socket

socket(family=AF_INET, type=SOCK_STREAM, proto=0, fileno=None)
```

- Optional Parameter `protocol`:
  - Selects the protocol used for the given socket type
  - Available protocols depend on OS, must implement same service
  - Often only one option is sensible, then can be set to `0` or omitted
- Optional Parameter `fileno`:
  - Creates a socket from the file descriptor, not discussed
- Return value:
  - If successful, a valid socket object is returned
- Exceptions:
  - Raises an `OSError` if an error occurs

Examples

```
from socket import socket
# All examples are for illustration only!
# To create a socket for TCP:
s = socket(AF_INET, SOCK_STREAM)
```

connect()

```
# 's' is a socket object
s.connect(address)
```

- Method of socket object; e.g. `s.connect(address)`
- For stream sockets (type `SOCK_STREAM` or `SOCK_SEQPACKET`) a client uses `connect()` to request establishment of a connection (e.g. TCP connection) with a server
- For datagram sockets (type `SOCK_DGRAM`) a client uses `connect()` to specify a default receiver for datagrams that is used when `s.send()` or `s.recv()` are used for data transmission

connect() (2)

```
# 's' is a socket object
s.connect(address)
```

- Parameter `address`
  - You have to prepare an address tuple containing the address / port number / etc. of the server or intended receiver
  - Format of this depends on address family
  - For IPv4 a tuple of `(address_ipv4, port)` would be used
- Exceptions:
  - Raises an `OSError` when an error occurs
  - Raises a `socket.gaierror` when an address error occurs
  - Raises a `TimeoutError` if a timeout is set and it expires

Examples

```
from socket import socket
# a TCP client
# get the address of a TCP server
address = ...
# create a socket s
s = socket (AF_INET, SOCK_STREAM)
# connect the socket to address of a TCP server
s.connect(address)
```

`send()` and Friends

```
# 's' is a socket object
amount = s.send(bytes, flags=0)
amount = s.sendto(bytes, address)
amount = s.sendto(bytes, flags, address)
```

- Socket object methods
- For all these functions the data that is to be sent is supplied in the `bytes` parameter, being a `bytes` or `bytearray` object
- `send()` requires that the addressee has been specified before by calling the socket's `connect()` method
- With `sendto()` the addressee has to be explicitly specified in the `address` parameter, which in case of the IPv4 address family will be an `(address_ipv4, port)` tuple
- There is rarely a reason to use the `flags` argument, under Linux see `man 2 send` for some options

`send()` and Friends (2)

```
# 's' is a socket object
amount = s.send(bytes, flags=0)
amount = s.sendto(bytes, address)
amount = s.sendto(bytes, flags, address)
```

- Return value:
  - The actual number of bytes (amount) placed in the socket buffer
- Exceptions:
  - Raises an `OSError` when an error occurs
  - In the case of `sendto()`, raises a `socket.gaierror` if an address error occurs
  - Raises a `TimeoutError` if a timeout is set and it expires

`send()` and Friends (3)

```
# 's' is a socket object
amount = s.send(bytes, flags=0)
amount = s.sendto(bytes, address)
amount = s.sendto(bytes, flags, address)
```

- When the underlying socket buffer still has enough space to accommodate data, these functions are non-blocking, but if there is not enough space, the caller is blocked
- An important note about the semantics:
  - When these functions return "successfully" you do not know more than that the data has been placed into the socket buffer
  - You cannot make any inference on whether or not the data has actually been transmitted or successfully received

Examples

```
from socket import socket
# a TCP client
# get the address of a TCP server
address = ...
# create a socket s
s = socket (AF_INET, SOCK_STREAM)
# connect the socket to address of a TCP server
s.connect(address)
# prepare a message msg, e.g. as bytearray
...
# send the message to the TCP server
amount = s.send(msg)
```

`recv()` and Friends

```
# 's' is a socket object
data = s.recv(bufsize, flags=0)
data, address = s.recvfrom(bufsize, flags=0)
```

- Socket object methods; `s` is a socket object.
- These functions can read received data from a socket
- In `recvfrom()` the caller wants more information about the sender, so the receiving protocol provides the address / port / etc. of the host from which data was received
- Parameter `bufsize`:
  - Caller specifies a buffer length which denotes the maximum amount of data read by this call
- These functions are normally blocking, but can be given options (in the `flags` parameter) to make them non-blocking
  - A timeout from `s.settimeout()` can be used too

`recv()` and Friends (2)

```
# 's' is a socket object
data = s.recv(bufsize, flags=0)
data, address = s.recvfrom(bufsize, flags=0)
```

- Return value(s) `data` and `address`:
  - The returned `data` is a `bytes` object containing the data received, its length is the number of bytes received
  - `address` is a tuple of the address information of the sender, its format as normal depends on the address family
- Exceptions:
  - Raises an `OSError` when an error occurs
  - Raises an `TimeoutError` if a timeout is set and it expires

Examples

```
from socket import socket
# a TCP client
# get the address of a TCP server
address = ...
# create a socket s
s = socket(AF_INET, SOCK_STREAM)
# connect the socket to address of a TCP server
s.connect(address)
# prepare a message msg
msg = ...
# send the message to the TCP server
amount = s.send(msg)

# receive a response from the server
resp = s.recv(4096)
```

bind()

```
# 's' is a socket object
s.bind(address)
```

- Links the socket to a particular IP address / port number combination
- Used by a socket object; e.g. `s.bind(addr)`
- `bind()` is non-blocking
- Parameter `address`:
  - Denotes the address to bind to
  - The format varies for each address family, a tuple of `(host, port)` for `AF_INET`
- Exceptions:
  - Raises an `OSError` when an error occurs

`bind()` – Address Formats

- The socket API is meant to support a whole range of underlying networking protocol stacks, not only IPv4, but also IPv6, Bluetooth, and others, and all of these have different address representations
- The `family` parameter of `socket()` indicates which protocol stack and address family (`AF_x`) you want to use
- For the IPv4 domain, we use a tuple of `(address_ipv4, port)`, where `address_ipv4` is the string of the hostname or IP address in dotted-decimal notation, and the `port` is an integer; an IPv4 of 0.0.0.0 allows the socket to be bound to all network interfaces.

Examples

```
from socket import socket
# a TCP server
# prepare the address
address = ...
# create a socket s
s = socket (AF_INET, SOCK_STREAM)
# link(bind) the socket to address
s.bind(address)
```

listen()

```
# 's' is a socket object
s.listen(backlog=...)
```

- `listen()` declares your willingness to accept incoming stream connections (TCP) of type `SOCK_STREAM` or `SOCK_SEQPACKET`, allocates resources like a queue for incoming connection requests
- It is only used on the server side
- `listen()` is non-blocking

`listen()` (2)

```
# 's' is a socket object
s.listen(backlog=...)
```

- Optional Parameter `backlog`:
  - There is a queue for yet **un-processed** incoming connection requests and `backlog` specifies how long this queue can be
  - Any excess connection request is declined (i.e. a packet is sent back to the client informing it of denial)
  - In Python this is optional, if omitted a default value is used
- Exceptions:
  - Raises an `OSError` when an error occurs

Examples

```
from socket import socket
# a TCP server
# prepare the address
address = ...
# create a socket s
s = socket (AF_INET, SOCK_STREAM)
# link(bind) the socket to address
s.bind(address)
#start listening
s.listen(5)
```

accept()

```
# 's' is a socket object
conn, address = s.accept()
```

- `accept()` waits for incoming connection requests of stream type (TCP), i.e. of type `SOCK_STREAM` or `SOCK_SEQPACKET`
- For an incoming connection a new socket is created (with the same port number as the socket on which previously `listen()` was called) and it is returned
- This is only used on a stream (TCP) server
- `accept()` is blocking (you can change that through options)

accept() (2)

```
# 's' is a socket object
conn, address = s.accept()
```

- Return values `conn` and `address`:
  - The socket `conn` whereby you can access the new connection
  - `address` gives you address information about the requesting client (e.g. its IP address and port number)
  - The precise type of the address information depends on the address family, for IPv4 it will be `(address_ipv4, port)`
- Exceptions:
  - Raises an `OSError` when an error occurs

Examples

```
from socket import socket
# a TCP server
# prepare the address, create socket s and bind it to address
address = ...
s = socket (AF_INET, SOCK_STREAM)
s.bind(address)

# start listening
s.listen(5)
...
# wait for incoming connection requests
# when a request is accepted, a new socket conn is created
# you will use conn to communicate with the client side
# the address of the client is stored in client_addr
conn,client_addr = s.accept()

# receive a message from the client and send a message back
msg = conn.recv(4096)
# prepare a response message resp
resp = ...
conn.send(resp)
```

close()

```
# 's' is a socket object
s.close()
```

- Just like for files, needs to be called whenever you do not need a socket anymore
- Reason: it gives back the socket resources (buffer, file descriptor) to the operating system for use by other applications

Examples

```
# continued
# close all the socket objects
conn.close()
s.close()
```

`bytes` and `bytearray` objects

- When working with packets, there are two Python binary sequence types to be aware of
- `bytes` objects are immutable sequences of bytes
  - `recv()` and `recvfrom()` return `bytes` object with data received
  - `str.encode()` (which you will see in the example later) returns encoded version of string as a `bytes` object
  - We use `str.encode`("UTF-8") and `str.decode`("UTF-8") in the examples
  - Article about Unicode: `https://docs.python.org/3/howto/unicode.html`
- `bytearray` objects are a mutable counterpart to `bytes`
  - When assembling packets, this mutability is useful, making it possible to update the values of individual bytes using indexing or slicing (much like a Python list)

Another Important Module

- As noted earlier regarding processes and blocking, `select()` can be used to prevent busy-looping and wait for input from multiple sockets at once
- Find out more about it in the `select` module: `https://docs.python.org/3/library/select.html`

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API
- Overview
- Main Socket API Functions
- Helpers

4 Socket Programming Examples

Memory Representation and Endianness

- Processors can use different ways to store data in memory
- This applies in particular to values that are relevant to Internet protocols, such as 16-bit port numbers or 32-bit IPv4 addresses
- Endianness is the byte order in data of multiple bytes in memory

A 16-Bit Example

- We are given a 16-bit integer number

$$b_{15}b_{14}b_{13}b_{12}b_{11}b_{10}b_9b_8b_7b_6b_5b_4b_3b_2b_1b_0$$

- Such a number must be stored somehow in memory, which we imagine as an ordered sequence of bytes
- Intel and AMD x86 processors use the *little-endian* format

$$b_7b_6b_5b_4b_3b_2b_1b_0 \qquad b_{15}b_{14}b_{13}b_{12}b_{11}b_{10}b_9b_8$$

  where the less significant byte sits at lower address and the more significant byte sits at higher address
- Some other processors use *big-endian*

$$b_{15}b_{14}b_{13}b_{12}b_{11}b_{10}b_9b_8 \qquad b_7b_6b_5b_4b_3b_2b_1b_0$$

- **Q**: Can you see the problem arising in a network?

Memory Representation and Endianness (2)

- To avoid confusion, for certain data types relevant for networking purposes (e.g. unsigned 16-bit or 32-bit integers), and in particular fields in a packet header, one:
  - standardizes / defines a *canonical representation* that is actually being transmitted – called the *network byte order*
  - asks hosts to convert between their own internal representation (*host byte order*) and network representation as needed
- In the Internet the canonical representation is big-endian

Helper Functions for Conversion

```
from socket import htonl, htons, ntohl, ntohs

netlong = htonl(hostlong)
netshort = htons(hostshort)
hostlong = ntohl(netlong)
hostshort = ntohs(netshort)
```

- These functions convert from host (h) to network (n) representation or vice versa
- They exist for 16 bit (short) and 32 bit (long) numbers
- They raise `OverflowError`s if given value can't fit in 16/32bits
- On machines where host and network byte order are identical, this is a no-op; otherwise, host performs a byte swap operation
- In Python these are used automatically by socket methods, but not all languages do this

getaddrinfo()

```
from socket import getaddrinfo

services = getaddrinfo(host, port, family=0, type=0, proto=0, flags=0)
```

- It is often necessary to determine how to connect to a host
- `getaddrinfo()` can provide information about what services a host supports
  - A service is the combination of the `family`, `type`, and `protocol` which can be used to create a socket able to connect to the host
- `getaddrinfo()` can also return IP address when given (textual) domain name, using the Domain Name Service (DNS)

`getaddrinfo()`(2)

```
from socket import getaddrinfo

services = getaddrinfo(host, port, family=0, type=0, proto=0, flags=0)
```

- Parameter `host`: a domain name or a string representation of IPv4/6 address
- Parameter `port`: a port number or a string service name
- Providing a `family`, `type`, and/or `protocol` will filter the returned services to only those using the given parameters
- Returns list of 5-tuples for each service:

  `[(family, type, proto, canonname, address), ...]`
  - Each tuple contains the `family`, `type`, and `protocol` of the service and the `address` to connect to it
  - The `address` will be in the format required for the `family`
  - For `AF_INET` families, the returned `address` will contain the IP address of the host provided by the DNS
  - `canonname` is the “standard” (canonical) name of the host

`getaddrinfo()`(3)

```
from socket import getaddrinfo

services = getaddrinfo(host, port, family=0, type=0, proto=0, flags=0)
```

- Exceptions:
  - Raises a `socket.gaierror` if the host cannot be found or the specified service isn't available for the host.
- Note: You may find code on the internet that uses `gethostbyname()` instead of `getaddrinfo()`, but `gethostbyname()` is deprecated

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API

4 Socket Programming Examples

Socket Programming Examples

- We will go through two examples:
  - A TCP server
  - A TCP client
- The code was ported to Python from the C code here:
  `http://www.linuxhowtos.org/C_C++/socket.htm`

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API

4 Socket Programming Examples
- A TCP Client
- A TCP Server

TCP Client – (1)

```
from sys import exit, argv # allows access to command-line arguments
from socket import socket, AF_INET, SOCK_STREAM, getaddrinfo
```

- Importing the required modules
  - `argv` from `sys` being a list of the command-line arguments
  - In programs you should use `sys.exit()` over `exit()`

TCP Client – (2)

```
def main():
  sock = None # define sock in case an error occurs before it is created

  try:
    if len(argv) != 3: # check number of command-line arguments
      # the 0th argument is the name of the file
      print(f"Usage:\n\n\tpython(3) {argv[0]} <hostname> <port>\n")
      exit()
    port = int(argv[2]) # convert the port to an integer
    # convert the string hostname from the command-line to an
    # IPv4 address using DNS name translation in getaddrinfo.
    # as we filter by AF_INET and SOCK_STREAM, the first service
    # will have the translated address
    services = getaddrinfo(argv[1], port, AF_INET, SOCK_STREAM)
    family, type, proto, canonname, address = services[0]
```

TCP Client – (3)

```
# create a new socket object, we omit protocol and fileno
# to use the default (TCP) and we are not using an existing
# file descriptor
sock = socket(AF_INET, SOCK_STREAM)

# connect to the TCP socket using the address tuple from
# getaddrinfo
sock.connect(address)
```

TCP Client – (4)

```
# get some data from the user to send
message = input("Please enter a message to send: ")
# convert the message into bytes
message = message.encode("UTF-8")

# send the data
amount = sock.send(message)
if amount < len(message):
  raise OSError("Unable to send whole message")

# read and display the response data
response = sock.recv(256)
print(response.decode("UTF-8"))
```

- Note: Python uses `UTF-8` for its strings, so we encode the message into the bytes representation of the string
  - Required as some `UTF-8` characters are longer than 1 byte

TCP Client – (5)

```
except ValueError: # raised by the int() conversion
  print(f"ERROR: Port '{argv[2]}' is not an integer")

except UnicodeDecodeError: # raised by .decode()
    print("ERROR: Response decoding failure")

except gaierror: # raised by address errors such as in getaddrinfo()
    print(f"ERROR: Host '{argv[1]}' does not exist")

except OSError as err: # raised by all socket methods
    print(f"ERROR: {err}") # print the error string
```

- Error handling is always important, here we catch the various errors that can occur from our previous actions

TCP Client – (6)

```
  finally: # reguardless of an exception or not, close the socket
    if sock != None:
      sock.close()


main()
```

- As with every external resource, close socket when we are done with it

Outline

1 Preliminaries

2 Sockets Fundamentals

3 The Socket API

4 Socket Programming Examples
- A TCP Client
- A TCP Server

TCP Server – (1)

```
from sys import exit, argv
from socket import socket, AF_INET, SOCK_STREAM


def main():
  sock = None
  conn = None

  try:
    if len(argv) != 2:
      print(f"Usage:\n\n\tpython(3) {argv[0]} <port>\n")
      exit()
    port = int(argv[1]) # convert the port to an integer

    # create the server socket
    sock = socket(AF_INET, SOCK_STREAM)
```

TCP Server – (2)

```
# bind the socket to the given port number and the
# ip address for any address.
# "0.0.0.0" or "" allows the socket to be bound to all local
# interfaces (there can be more than one). If only one
# interface is desired then use the address of that interface
sock.bind(("0.0.0.0", port))

# start listening for requests on the socket, allowing
# a max of 5 queued connection requests
sock.listen(5)

# wait for incoming connection requests (blocking)
# the client address and port is stored in 'client'
# and the new socket for the connection is in 'conn'
conn, client = sock.accept()
```

TCP Server – (3)

```
# read the received data and display it
# note the use of the conn socket not the server
message = conn.recv(256)
message = message.decode("UTF-8") # decode the bytes into a string
print(f"Here is the message: {message}")

# respond with our message
response = "I got your message".encode("UTF-8")
conn.send(response)
```

- As we encoded the string sent from the client, we decode it here
- This ensures any multi-byte characters are correctly read

TCP Server – (4)

```
  except ValueError: # port int conversion
    print(f"ERROR: Port '{argv[2]}' is not an integer")

  except UnicodeDecodeError: # message decode
    print("ERROR: Decoding/encoding failure")

  except OSError as err: # raised by all socket methods
    print(f"ERROR: {err}") # print the error string

  finally: # close the sockets
    if sock != None:
      sock.close()
    if conn != None:
      conn.close()


main()
```

TCP Server – (5)

- Now suppose you save your server program as `server.py` and your client program as `client.py`, and you want to run your server at port 1234
- You can open two linux terminals
- In one terminal you start the server by using the command:

```
$ python3 server.py 1234
```

- In the other terminal you start the client by using the command:

```
$ python3 client.py localhost 1234
```

[1] Jon C. R. Bennett, Craig Partridge, and Nicholas Shectman.
Packet Reordering is Not Pathological Network Behaviour.
IEEE/ACM Transactions on Networking, 7(6):789–798, December 1999.